

HireIQ Recruiter OS

COMPLETE TECHNICAL & OPERATING COST REFERENCE MANUAL

HireIQ Recruiter OS is a comprehensive Applicant Tracking System (ATS) engineered to streamline and automate human resource workflows. By combining local vector databases, local open-source machine learning models, and secure cloud AI completion layers, HireIQ delivers a high-capacity, responsive, and privacy-respecting platform at near-zero operational costs.

1. System Overview & Technology Stack

The system is structured as a decoupled client-server architecture. The frontend is built in React (using Vite for development), serving as a Single Page Application. The backend runs on FastAPI, serving asynchronous REST endpoints and streaming connections via Server-Sent Events (SSE).

Component	Technology	Operational Purpose
Frontend	React 18 + Vite	Fast, modular UI; handles local proctoring feeds.
Backend API	FastAPI (Python)	Asynchronous, fast REST execution, autodocs.
Database	MongoDB Atlas (Cloud)	Flexible schema matching parsed candidate structures.
Cloud Storage	Supabase Storage	Secure storage for resume documents and audio recordings with local fallback.
AI Completion	Groq (LLaMA-3.3-70b)	Sub-second parsing of resumes and job descriptions.
Speech-to-Text	Groq Whisper (v3-turbo)	Translates interview session audio to transcripts.
Embeddings Engine	SentenceTransformers (Local)	Generates 384-dim semantic vectors on the host CPU.
Vector DB	FAISS Flat Index	Runs fast in-memory semantic matches without external APIs.
Video Call Engine	LiveKit Integration	Modern, native WebRTC conferencing with secure token access control.
Proctoring Tracking	MediaPipe FaceMesh	Runs client-side gaze/attention checks in JS.
Email Relay	SMTP (Gmail app)	Sends verification codes and meeting invitations.

2. Operating Cost Estimation & Free-Tier Capacity

HireIQ is engineered to run on a **zero-dollar monthly operating cost** for development and small-scale operations. By choosing components that provide generous free limits, you can operate the system without inputting any billing credentials.

How Many Resumes Can Be Screened For Free?

The absolute limiting factor in the free stack is the **MongoDB Atlas M0 Free Tier** which provides **512 MB** of storage. An average candidate record (comprising parsed education, work history, skills lists, FAISS embedding indices, and Groq-generated feedback narratives) takes approximately **15 KB** of storage space in MongoDB.

$512 \text{ MB} * 1024 \text{ KB/MB} / 15 \text{ KB} = \sim 34,952 \text{ Candidates.}$

Accounting for index growth and system overhead, the database can comfortably hold **between 25,000 and 30,000 resumes** completely for free.

Groq Cloud Free Limits: Groq provides 100,000 requests per day (RPD) and 180,000 tokens per minute (TPM) on its developer free tier. This translates to **3,000,000 resume parses or chatbot operations per month**, which far exceeds the database storage threshold.

Cost Table If Scaling Beyond Free Limits:

Service	Free Limit	Billing Rate (Paid Tier)	Estimated Cost for 10,000 Re
Groq LLaMA-3.3	100k requests/day	\$0.59 / million input tokens	~\$5.00 total
Groq Whisper	100k requests/day	\$0.03 / hour of audio	~\$5.00 total
MongoDB Atlas	512 MB Storage	\$9.00 / month (2 GB shared)	\$9.00 / month flat
FAISS & Embeddings	Unlimited (Local)	\$0.00 (Self-Hosted on CPU)	\$0.00
Supabase Storage	1 GB Free Tier	\$0.021 / GB of stored data	~\$0.10 total
LiveKit Cloud	50 GB bandwidth/mo	\$0.05 / GB bandwidth over limit	\$0.00 (under 50GB)
SMTP Email	500 emails/day	\$0.00 (Gmail server limit)	\$0.00

3. Step-by-Step System Flow Operations

3.1 Recruiter Authentication

Recruiters sign in or register through custom routes. Passwords are salted and hashed using **bcrypt** in the backend before database insertion. Upon successful validation, a JWT token is generated by python-jose with a 24-hour expiration time. The token is sent to the client, stored in localStorage, and passed in the headers (Authorization: Bearer <token>) of all subsequent API requests. Router guards on the frontend automatically redirect unauthenticated users back to the Login screen.

3.2 Job Posting & Requirement Parsing

The recruiter drafts a job description (JD) in the creation panel and submits it. The backend maps the raw text and sends it to the Groq API using the LLaMA-3.3-70b model with a strict Pydantic JSON template. The LLM extracts required skills, preferred skills, minimum experience, certifications, and domain requirements. If Groq is down or unreachable, a regex-driven fallback scanner automatically parses the text locally and extracts skills using RapidFuzz mappings to prevent user failure.

3.3 Resume Upload & Parsing Cascade

When a resume is uploaded (PDF or DOCX), it is saved in the server's local uploads/ directory. Raw text is extracted using PyMuPDF or mammoth. The parsing cascade first runs a fast, rule-based keyword scan for emails, phone numbers, names, and skills. Next, the raw text is sent to Groq LLaMA-3.3 to build a zero-hallucination JSON payload of employment history, degree details, certifications, soft skills, and confidence metrics. The resulting document is stored in MongoDB. Concurrently, the text is vectorized locally via SentenceTransformers (all-MiniLM-L6-v2) and synced to the local in-memory FAISS database.

3.4 Heuristic Scoring & Ranking Engine

The matching engine calculates compatibility scores deterministically, ensuring high speed and protection against AI hallucinations. The formula splits weights: **40% Skills Match** (exact name match + alias lookup + Levenshtein distance checks), **25% Experience Match** (compares candidate work history timeline against JD requirements), **15% Semantic Match** (measures cosine distance using the FAISS index), **10% Projects Match**, **5% Certifications**, and **5% Profile Completeness**. Hard deductions are applied afterwards (e.g., -20 points if >70% of required skills are missing). Groq is subsequently used to translate these numbers into an executive summary narrative, highlighting strengths and concerns without determining the numeric score directly.

3.5 Recruiter Copilot (Chatbot)

The recruiter can ask questions to query candidates and jobs in natural language. A classification prompt evaluates the recruiter's intent. If the question relates to candidate details or statistics, FastAPI queries the MongoDB Atlas collection, compiles the relevant context, and feeds it to Groq LLaMA-3.3. The chatbot streams the response back using Server-Sent Events (SSE) for real-time responsiveness. Session history stores the last 10 dialog turns for coherent back-and-forth conversations.

3.6 Interview Scheduling & Reminders

The recruiter selects a candidate and schedules an interview slot. The backend checks for timing conflicts. It connects to the LiveKit API to initialize a video room and issues secure, cryptographically signed participant tokens. The candidate join link is built: {FRONTEND_URL}/candidate-interview/{token}. The candidate details are saved to MongoDB, and emails are dispatched via SMTP (Gmail App passwords). The backend runs a scheduled thread (APScheduler) that sends reminder notifications 15 minutes before the start time.

3.7 Candidate Interview, MediaPipe Proctoring & Audio transcription

The candidate joins via the token link. The browser resolves the LiveKit token and connects natively via WebRTC. A WebRTC Voice Activity Detector measures speaking/listening ratios, while MediaPipe FaceMesh tracks face presence and eye vectors to flag if the candidate looks away from the screen. JavaScript handlers log copy/paste and tab-switching events. Audio streams are captured, and once the call is terminated, transmitted to Groq Whisper for speech-to-text conversion. The transcript is processed by Groq LLaMA-3.3 alongside proctoring violations to compile a scorecard of communication, confidence, technical ability, and cheating risks, and builds a printable PDF report.